Plan Workflow Technical Architecture

Target design for migrating plan generation to LangGraph now, while preserving clean entry points for chat, readiness surveys, and workout-performance adaptation later.

BOUNDARY

Use one reusable **plan graph**, not one application-wide graph. API routes, the chat graph, survey handlers, and workout logging invoke it through typed commands. The plan graph owns programming decisions; services own I/O.

System boundary

Entry adapters	Plan graph	Shared services	Persistence
Plan endpoint Chat delegation Survey handler Workout logger	Route command Load context Rewrite/search/adapt Generate + validate Return proposal	search.py training_history.py workouts.py profile.py	Supabase queries transaction/RPC optimistic version check confirmed write

Graph topology

The first release implements only the plan-page branch; the other routes are extension points.

Origin	Route	Shared tail
plan_page	rewrite -> search -> generate_full	validate -> check_conflicts -> finalize
chat	conditional_search -> generate_patch	validate -> check_conflicts -> finalize
readiness_survey	adapt_fast (no rewrite/search)	validate -> check_conflicts -> finalize
workout_completion	evaluate_targets -> adapt_fast	validate -> check_conflicts -> finalize

Immediate graph

Parallelize independent context loading and query preparation once correctness is established.

```
START -> load_context ------------------------\
                                               -> generate_full -> validate -> check_conflicts -> END
START -> rewrite -> search --------------------/
```

LangGraph fan-in can be expressed with add_edge(["load_context", "search"], "generate_full"). Both branches must write distinct state keys, or use explicit reducers for shared keys.

Contracts, State, and Runtime Context

Keep durable workflow data, request-scoped credentials, and public API models separate.

Invocation contract

```
class PlanningCommand(BaseModel):
    origin: Literal[
        "plan_page", "chat", "readiness_survey", "workout_completion"
    ]
    operation: Literal["create", "modify", "adapt"]
    request: str | None = None
    target_workout_ids: list[UUID] = Field(default_factory=list)
    survey: ReadinessSurvey | None = None
    completed_workout_id: UUID | None = None
    expected_versions: dict[UUID, datetime] = Field(default_factory=dict)
```

State versus context

PlanState	WorkflowContext

`class PlanState(TypedDict, total=False):`
`  command: PlanningCommand`
`  rewritten_request: str`
`  user_profile: UserTrainingContext`
`  training_history: list[TrackedWorkoutSummary]`
`  existing_workouts: list[PlannedWorkout]`
`  sources: list[Source]`
`  draft_plan: WorkoutPlan`
`  draft_patch: WorkoutPatch`
`  validation_errors: list[str]`
`  conflict: PlanConflict`
`  result: PlanningResult`
	`@dataclass(frozen=True)`
	`class WorkflowContext:`
	`  user_id: str`
	`  access_token: str`
	`  request_id: str`
	`  conversation_id: str | None = None`

- State contains values produced or consumed by nodes and may later be checkpointed.
- Context contains authentication and request identity. Do not copy the JWT into state, prompts, traces, or model input.
- A planning graph does not need MessagesState. The chat graph should normalize conversation into PlanningCommand before delegation.

Result contract

```
class PlanningResult(BaseModel):
    status: Literal["proposal", "conflict", "rejected"]
    plan: WorkoutPlan | None = None
    patch: WorkoutPatch | None = None
    conflict: PlanConflict | None = None
    requires_confirmation: bool = True
    rationale: str
```

Return a proposal, not an implicit database mutation.

The API or chat surface asks for confirmation. A separate write command persists the accepted version.

Model ownership

Location	Owns
ai/contracts/workouts.py	PlannedWorkout, PlannedExercise, TrackedWorkout, CompletedSet, optional RepRecord
ai/workflows/plan/contracts.py	PlanningCommand, WorkoutPatch, PlanConflict, PlanningResult
ai/workflows/plan/state.py	PlanState and WorkflowContext only
api/schemas.py	Public request/response transport models; map to domain contracts at the router boundary

Node and Service Architecture

Nodes orchestrate. Services perform external I/O. Contracts define the boundary between them.

Node responsibilities

Node	Reads	Writes	Rule
load_context	command + context	profile, history, existing workouts	Parallelize independent DB reads; enforce user scope in every query.
rewrite	command.request	rewritten_request	Low-latency model; plan-oriented phrasing only, no training decisions.
search	rewritten_request	sources	Call shared search agent; preserve DOI/URL attribution and source limit.
generate_full	request, context, sources	draft_plan	Structured output bound to WorkoutPlan; no DB writes.
generate_patch	request + target workouts	draft_patch	Change only identified workouts/exercises; include rationale.
adapt_fast	survey or performance signals	draft_patch	Fast model, minimal context, no search by default.
validate	draft plan/patch	errors or normalized draft	Deterministic domain checks first; at most one bounded repair call.
check_conflicts	draft + existing workouts	PlanConflict	Never silently overwrite future work.
finalize	draft/errors/conflict	PlanningResult	Stable public workflow result; requires confirmation for mutations.

Service boundary and call direction

```
FastAPI lifespan
    plan_graph = build_plan_workflow()
    app.state.plan_graph = plan_graph
    app.state.chat_graph = build_chat_workflow(plan_graph)

plan router
    -> plan_graph.ainvoke({"command": command},
context=context)
        -> plan nodes -> typed services -> db/supabase.py

chat planning node
    -> injected plan_graph (same compiled instance)
```

No internal HTTP calls

Endpoints and workflows call the same typed services. Do not call your own FastAPI routes from a graph.

Inject the compiled graph

Build chat with the compiled plan graph as a dependency. Do not make graph nodes reach into FastAPI app.state, and do not store graph objects in PlanState.

Suggested package shape

```
ai/
  contracts/workouts.py
  services/
    search.py
    training_history.py
    workouts.py
  workflows/plan/
    contracts.py
    graph.py
    prompts.py
    state.py
    nodes/
      load_context.py rewrite.py search.py
      generate.py validate.py check_conflicts.py
```

Persistence Safety and Migration Plan

Keep generation reversible and observable; make writes explicit, authorized, and concurrency-safe.

Planned versus tracked workout schemas

Planned domain	Tracked domain
Workout date + intent Exercise order Compact sets/reps target Target load, distance, duration, RPE Rest + notes	Workout execution status CompletedExercise CompletedSet per set Actual reps/load/RPE/pain Optional RepRecord[] only when a value varies by rep

Write protocol

1	**Read**	Load target workouts with updated_at/version under the authenticated user.
2	**Propose**	Graph returns WorkoutPlan or WorkoutPatch plus expected versions. No write yet.
3	**Confirm**	UI presents create/replace/merge/cancel when future workouts conflict.
4	**Commit**	Supabase transaction/RPC verifies ownership and expected version, then applies all rows atomically.
5	**Report**	Return saved identifiers and new versions; reject stale proposals with a conflict response.

First migration slice

Phase	Implement	Exit condition
1. Contracts	Move shared workout models out of state.py; define command/state/result contracts.	Models validate lifting, running, and plyometric plans.
2. Nodes	Implement load_context, rewrite, search, generate_full, validate.	Each node has focused unit tests and typed state updates.
3. Graph	Wire only origin=plan_page; compile once in FastAPI lifespan.	Graph returns a valid PlanningResult for representative prompts.
4. Endpoint	Adapt current plan route to invoke the graph; preserve public response shape.	Frontend behavior remains unchanged.
5. Observe	Tag nodes in LangSmith; record latency, tokens, tool rounds, validation failures.	Baseline is stable enough to compare future branches.
6. Extend later	Add survey, completion, and chat adapters one at a time.	Shared validation and persistence code stays unchanged.

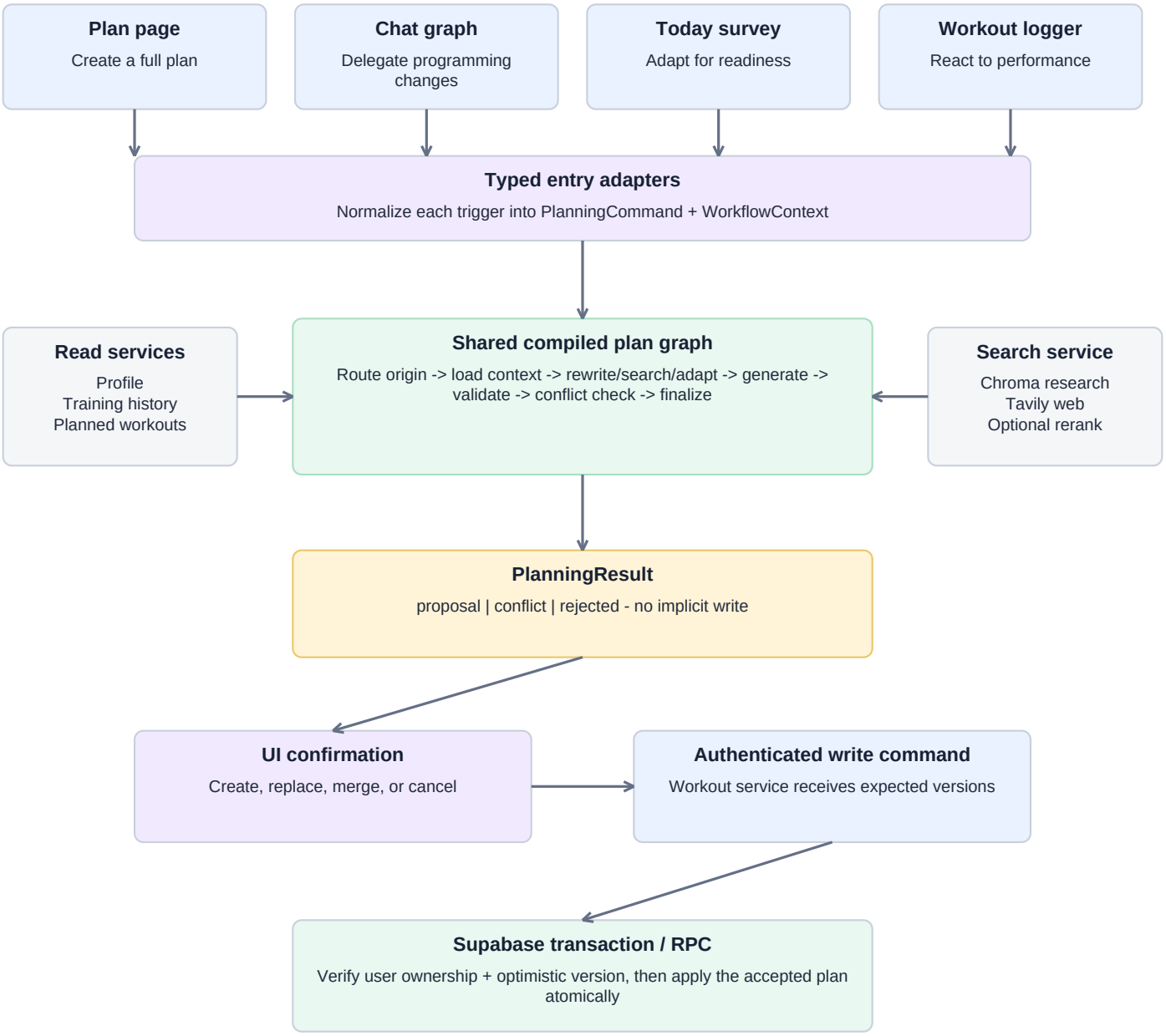
CURRENT SCAFFOLD NOTE

In plan/state.py, validation fields should use pydantic.Field, not dataclasses.Field. Keep domain workout models outside state.py so API, chat delegation, and tracking can reuse them without importing workflow internals.

Final Workflow: Entry Points and Ownership

All planning triggers use one compiled plan graph. The graph proposes changes; authenticated services persist confirmed results.

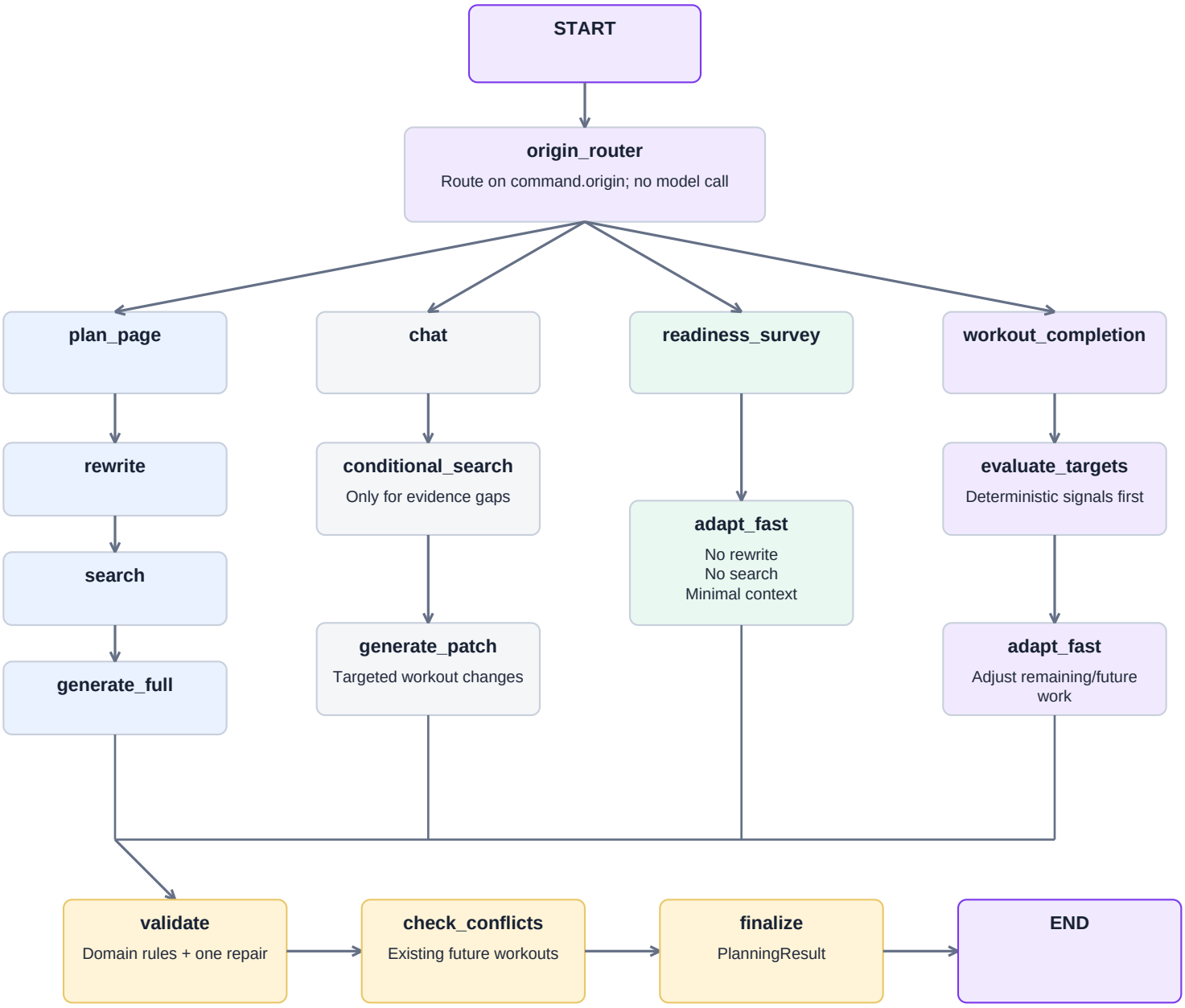
PRODUCT ENTRY POINTS



Chat retains its own conversation and search behavior. It invokes planning only for programming decisions; simple database reads or mechanical mutations can remain allowlisted Supabase tools.

Internal Plan Graph

The origin router is deterministic. Expensive rewrite, search, and full generation are used only by the routes that need them.



IMPLEMENT FIRST: plan_page only. Keep origin and operation in the command contract, but defer the other three branches until the migrated path is stable and measured in LangSmith.